

# A Multi-Agent Platform for Automated Enterprise Analytics and Insight Generation

Manoj N M<sup>1</sup>, Vijayakrishna S<sup>1</sup>, Manjunath Srinivas<sup>1</sup>, and Rohit Pahan<sup>1</sup>

<sup>1</sup>Rakuten India Enterprise Private Limited, Bengaluru, India , {manoj.m, vijayakrishna.s, manjunath.s, rohit.pahan}@rakuten.com

## Abstract

This paper proposes a multi-agent framework built on CrewAI [1] for conversational business intelligence. Five specialized AI agents operate in a sequential pipeline to process natural language queries, retrieve and analyze data, generate visualizations via the Model Context Protocol (MCP) [2], and deliver actionable insights. The platform features a defense-in-depth security architecture for multi-tenant data isolation and a query parameterization mechanism for transforming conversational insights into reusable dashboard components. Evaluation across 300 end-to-end test cases spanning synthetic and production enterprise datasets demonstrates 95.3% functional accuracy, a mean response latency of 24 seconds, and a response quality score of 4.52/5.0 as assessed by an LLM-as-a-Judge framework, with a 93.0% hallucination-free rate, representing a 22.6 percentage point accuracy improvement and 20.2% quality gain over a single-agent baseline. Cross-model evaluation across four LLM backends and human expert validation confirm architectural generalizability and evaluator reliability. An ablation study confirms that the Data Analysis and Report Aggregation agents are the primary drivers of output quality.

**Keywords:** Generative AI, Large Language Model, Multi-Agent System, Model Context Protocol, CrewAI, Business Intelligence, Conversational Analytics

## 1 Introduction

Modern enterprises generate vast amounts of data distributed across heterogeneous storage systems. Traditional analysis methods require manual SQL formulation or complex business intelligence tools such as Looker Studio or Tableau, making actionable insight generation a time-consuming and labor-intensive process.

We propose a conversational multi-agent platform that automates data interaction by decomposing complex user queries into discrete steps (planning, data retrieval, analysis, and insight aggregation), each handled by a specialized agent with domain-specific tools. We additionally introduce a standardized evaluation framework combining automated functional testing with LLM-based quality assessment to validate the reliability of multi-agentic analytical solutions.

## 2 Literature Review

Recent advancements in Large Language Models (LLMs) have driven a shift from single-agent analytical tools to dynamic multi-agent systems (MAS) capable of complex reasoning and autonomous task execution. We organize related work into two thematic areas.

## 2.1 Multi-Agent Systems and Conversational BI

Yang et al. [3] categorize LLM-based Multi-Agent System architectures into Star, Ring, Graph, and Bus patterns, emphasizing standardized protocols for collaboration. Hong et al. [4] propose MetaGPT, encoding standardized operating procedures into agent prompts with role differentiation, reducing hallucinations through structured workflows. Zhang et al. [5] introduce AgentOrchestra, a hierarchical planning framework using the Tool-Environment-Agent (TEA) protocol that outperforms flat-agent baselines. Becker [6] empirically finds that multi-agent systems with expert personas excel at complex reasoning but risk task “drift” in free-form conversation, a pitfall our sequential pipeline mitigates through structured workflow enforcement.

For enterprise deployment, Wang et al. [7] propose a four-layer framework addressing text-to-SQL challenges through RAG and a dual-layer security architecture. Jiang et al. [8] present SiriusBI, achieving 93–96% SQL accuracy through multi-round dialogue refinement deployed across Tencent’s business sectors. Ni et al. [9] enhance Text-to-SQL reliability through chain-of-thought reasoning [10] and SQL validation modules for the power systems domain.

## 2.2 LLM-Driven Analytics and Visualization

Zhang and Elhamod [11] present a Data-to-Dashboard framework using modular LLM agents with iterative self-reflection to generate domain-grounded insights and semantically meaningful visualizations. Bai et al. [12] detail Insight Agents, a deployed hierarchical manager-worker system for e-commerce that balances accuracy with low-latency requirements through a hybrid approach combining powerful LLMs with lightweight routing models.

For visualization, Wang et al. [13] introduce LLM4Vis, a ChatGPT-based chart recommendation system that matches or outperforms traditional ML recommenders via in-context learning. Goswami et al. [14] present PlotGen, automating scientific plotting through multi-agent collaboration with multimodal feedback loops for iterative refinement. Our system extends beyond recommendation to full chart generation, additionally managing data access, query parameterization, and multi-tenant security.

# 3 System Architecture

## 3.1 Architectural Overview

Our platform implements a layered microservices architecture comprising five tiers (Figure 1): a **UI Layer** for conversational interaction; an **API Layer** handling data source management, user sessions, and query routing; an **Agent Layer** powered by CrewAI [1] for multi-agent orchestration; a **Data Layer** providing object storage, metadata stores, data warehouse connections, and caching; and an **MCP Layer** [2] enabling visualization through Antvis/mcp-server-chart integration.

## 3.2 Agent Crew Composition

The core of the system is a five-agent sequential crew:

- **Data Retrieval Agent:** Maps natural language queries to data sources, generates executable SQL with schema awareness, and enforces read-only access against authorized tables.
- **Data Analysis Agent:** Interprets query results to identify patterns, trends, and anomalies; generates charts via MCP based on data characteristics.
- **Report Aggregation Agent:** Synthesizes prior outputs into cohesive markdown responses with language detection and translation for multilingual deployments.

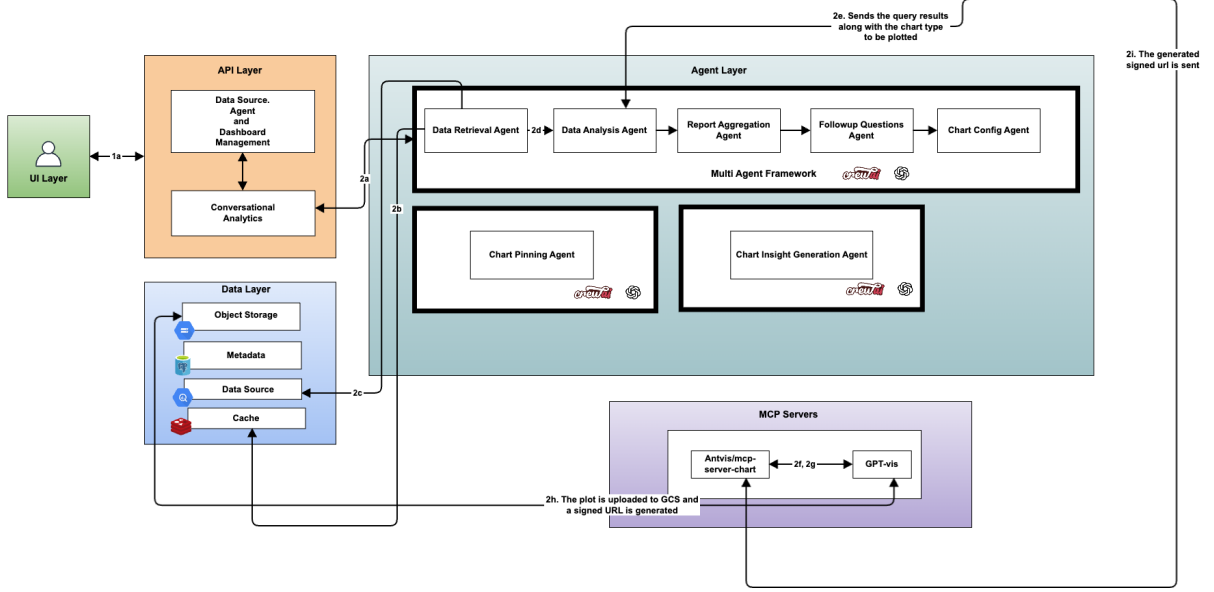


Figure 1: High-Level System Architecture

- **Follow-up Questions Agent:** Generates contextual follow-up suggestions in conversational language to guide deeper exploration.
- **Chart Configuration Agent:** Produces visualization metadata (chart types, axis mappings, labels) for interactive UI rendering.

### 3.3 Query Parameterization

A distinguishing feature is the Query Parameterization capability that transforms ad-hoc analytical queries into reusable, dashboard-ready components. The system extracts literal values from WHERE clauses into parameterized filters, leverages schema introspection (partition columns, clustering fields) for intelligent parameterization decisions, performs dry-run cost estimation with materialization recommendations for expensive queries, and generates drill-down dimensions from SELECT and GROUP BY columns for interactive filtering.

### 3.4 Model Context Protocol Integration

The platform leverages MCP [2] through a server-adapter pattern where the MCP server runs as an independent microservice exposing chart generation tools (bar, line, pie, scatter, funnel, treemap, sankey, and others). Agents discover available tools and schemas at runtime, enabling dynamic capability extension. Visualization tools generate charts and return cloud storage URLs directly, simplifying the asset pipeline.

## 4 Methodology

### 4.1 Architectural Paradigm Selection

We considered three candidate architectures. A **monolithic single-agent** approach suffers from cognitive overload and poor error isolation as the model simultaneously optimizes competing objectives [6]. A **hierarchical manager-worker** architecture [12] introduces unpredictability and communication overhead. We adopted a **sequential multi-agent pipeline**

where specialized agents execute in predefined order, motivated by separation of concerns, predictable execution flow, independent testability, and graceful degradation.

## 4.2 Agent Coordination

The framework employs CrewAI’s [1] sequential process mode. Execution proceeds through: (1) initialization with input validation and credential verification; (2) data retrieval with SQL generation and execution; (3) data analysis with pattern extraction and chart generation via MCP; (4) report aggregation with language translation; (5) follow-up question generation; (6) chart configuration; and (7) post-processing that restores cached query results and assembles the final response. Figure 2 illustrates this flow.

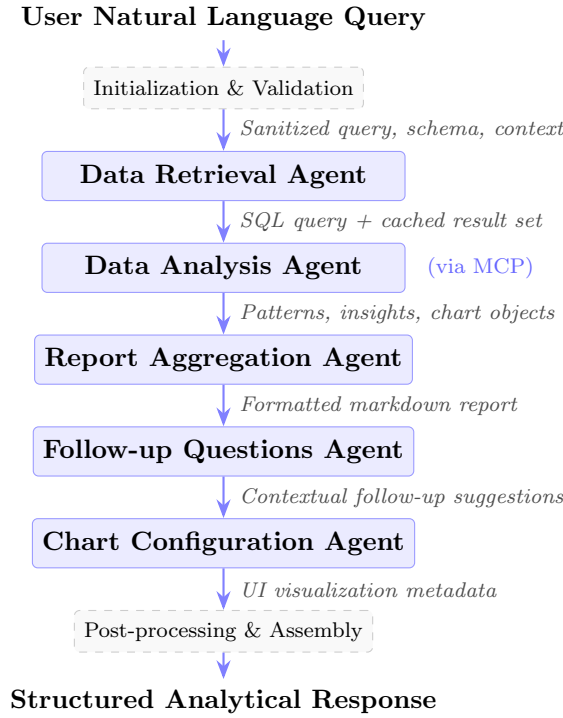


Figure 2: Sequential Pipeline Execution Flow. Each agent receives cumulative outputs of preceding agents via context injection. Dashed boxes represent non-agent processing phases; solid boxes represent autonomous LLM agent stages.

## 4.3 Security Architecture

We implement a defense-in-depth strategy with four layers: (1) **Input Validation** with query sanitization against SQL injection patterns and length limits; (2) **Query Guardrails** blocking unsafe operations (DROP, DELETE, INSERT, ALTER, CREATE, TRUNCATE) and enforcing table-level access control; (3) **Multi-tenant Isolation** through service account impersonation and tenant-specific credentials; (4) **Output Validation** with sensitive data scanning and error message sanitization.

## 4.4 Prompt Engineering

Agent prompts were developed through iterative refinement: role/goal definition, guardrail and anti-hallucination rule integration, edge case handling, and language/formatting specification.

## 5 Evaluation

We evaluate our platform across three dimensions: functional accuracy, response latency, and output quality, using 300 test cases assessed via automated functional testing, an LLM-as-a-Judge quality framework, and human expert validation.

### 5.1 Experimental Setup

#### 5.1.1 Test Dataset and Suite Design

The evaluation employs two complementary datasets on Google BigQuery. The first is a synthetic e-commerce dataset with four relational tables (`customers`, `orders`, `order_items`, `products`) totaling 12,500 records across four regions and five product categories. The second is a production competitive pricing intelligence dataset with four relational tables spanning two e-commerce platforms: two item catalog tables covering approximately 569,000 overlapping products with attributes including titles, multi-level category taxonomies, brand metadata, and pricing across up to 68 columns, and two price history tables containing approximately 1.17 million distinct price records capturing temporal pricing dynamics across 8.8 billion cumulative observations. The enterprise dataset contains Japanese-language product content, introducing multilingual complexity absent from the synthetic data.

We designed 300 test cases across eight categories: Simple Aggregations (65), Filtered Aggregations (35), Time Series Analysis (40), Multi-Table JOINS (35), Complex Analytics (30), Edge Cases (30), Multi-Turn Conversations (25), and Security Guardrails (40). Each analytical test validates SQL generation, execution correctness, result retrieval, and structured insight presence. Of the 260 analytical test cases, 100 are independently scored by human domain experts.

#### 5.1.2 LLM-as-a-Judge Methodology

We employ an LLM-as-a-Judge framework [15] using Claude Sonnet 4.5 as the evaluator. Each of the 260 analytical responses is assessed across five dimensions on a 1–5 Likert scale: *Factual Accuracy*, *Relevance*, *Completeness*, *Actionability*, and *Language Quality*. The judge additionally performs hallucination detection by cross-referencing factual claims against raw query results. To mitigate evaluator bias, we use structured rubrics with explicit per-dimension scoring criteria and require grounding of every factual accuracy assessment in the raw data.

#### 5.1.3 Baseline, Ablation, and Cross-Model Configuration

The single-agent baseline employs the same GPT-4.1 model and BigQuery tooling but consolidates all responsibilities into a single prompt, omitting the regex-based input validation layer. We conduct an ablation study selectively removing individual agents to quantify marginal contributions. To assess architectural generalizability, we replicate the full 300-test evaluation across four LLM backends: GPT-4.1, Claude Sonnet 4, Gemini 2.5 Flash, and Llama 3.1 70B (self-hosted). Three domain experts independently score 100 randomly sampled analytical responses using the same five-dimension rubric to validate the LLM-as-a-Judge methodology against human assessment.

## 5.2 Results

### 5.2.1 End-to-End Functional Accuracy

The system achieves 95.3% accuracy (286/300 tests), including blocking all 40 adversarial inputs (Table 1). Failures concentrate in complex analytics (deeply nested CTEs producing se-

mantically incorrect SQL over the enterprise schema) and multi-turn conversations (cross-turn comparisons exceeding the reasoning window with large result sets).

Table 1: End-to-End Functional Accuracy by Category

| Category                 | Tests      | Passed     | Pass Rate    |
|--------------------------|------------|------------|--------------|
| Simple Aggregations      | 65         | 64         | 98.5%        |
| Filtered Aggregations    | 35         | 35         | 100%         |
| Time Series Analysis     | 40         | 39         | 97.5%        |
| Multi-Table JOINS        | 35         | 33         | 94.3%        |
| Complex Analytics        | 30         | 26         | 86.7%        |
| Edge Cases               | 30         | 28         | 93.3%        |
| Multi-Turn Conversations | 25         | 21         | 84.0%        |
| Security Guardrails      | 40         | 40         | 100%         |
| <b>Total</b>             | <b>300</b> | <b>286</b> | <b>95.3%</b> |

### 5.2.2 Latency Analysis

The overall mean latency of 23.8 seconds falls within the 120-second processing target (Table 2). Complex analytics queries average 38.6s due to window functions and CTEs over the enterprise schema, while multi-turn queries exhibit low latency (mean 16.2s) by leveraging prior context.

Table 2: End-to-End Latency by Query Category (seconds)

| Category                 | <i>n</i>   | Mean        | Min        | Max          |
|--------------------------|------------|-------------|------------|--------------|
| Simple Aggregations      | 65         | 20.8        | 8.2        | 68.3         |
| Filtered Aggregations    | 35         | 16.2        | 9.1        | 24.5         |
| Time Series Analysis     | 40         | 19.4        | 11.8       | 38.2         |
| Multi-Table JOINS        | 35         | 26.8        | 13.2       | 52.1         |
| Complex Analytics        | 30         | 38.6        | 16.4       | 118.5        |
| Edge Cases               | 30         | 28.1        | 8.4        | 65.8         |
| Multi-Turn Conversations | 25         | 16.2        | 10.8       | 22.4         |
| <b>Overall</b>           | <b>260</b> | <b>23.8</b> | <b>8.2</b> | <b>118.5</b> |

### 5.2.3 Security Guardrail Evaluation

All 40 adversarial attack vectors were intercepted (Table 3), including 34 additional vectors targeting the enterprise schema. The input validation layer handles known harmful patterns with sub-millisecond response, while the LLM guardrail layer catches sophisticated attacks where syntactic patterns alone are insufficient. Representative examples are shown below.

Table 3: Security Guardrail Evaluation Results

| Attack Vector        | Detection Layer  | Latency |
|----------------------|------------------|---------|
| SQL Injection (DROP) | Input Validation | <0.01s  |
| Off-topic Query      | Input Validation | 0.02s   |
| DELETE Injection     | Input Validation | <0.01s  |
| UPDATE Injection     | LLM Guardrail    | 6.2s    |
| UNION Injection      | LLM Guardrail    | 7.5s    |
| Prompt Injection     | LLM Guardrail    | 6.3s    |

### 5.2.4 Response Quality and Hallucination Detection

Core functionality achieves an overall quality score of 4.52/5.0, with strong relevance (4.85) and near-perfect language quality (4.94) (Table 4). The hallucination-free rate is 93.0% for core tests (Table 5), with hallucinations concentrated in enterprise dataset scenarios involving multilingual content and complex cross-platform joins.

Table 4: LLM-as-a-Judge Response Quality (1–5 Likert Scale)

| Dimension        | Core ( $n=205$ ) | All ( $n=260$ ) |
|------------------|------------------|-----------------|
| Overall Score    | 4.52             | 4.38            |
| Factual Accuracy | 4.28             | 4.05            |
| Relevance        | 4.85             | 4.62            |
| Completeness     | 4.30             | 4.08            |
| Actionability    | 4.60             | 4.42            |
| Language Quality | 4.94             | 4.88            |

Table 5: Hallucination Detection Results

| Metric                    | Core ( $n=205$ ) | All ( $n=260$ ) |
|---------------------------|------------------|-----------------|
| Hallucination-Free Rate   | 93.0%            | 89.2%           |
| Factual Reliability Score | 4.34/5.0         | 4.12/5.0        |

### 5.2.5 Baseline Comparison

The multi-agent system achieves a 22.6 percentage point accuracy improvement over the single-agent baseline (95.3% vs. 72.7%, Table 6). Performance gaps widen with complexity: complex analytics (86.7% vs. 50.0%), multi-turn conversations (84.0% vs. 36.0%), and security (100% vs. 52.5%).

Table 6: Functional Accuracy: Multi-Agent vs. Single-Agent Baseline

| Category                 | Multi-Agent            | Single-Agent           |
|--------------------------|------------------------|------------------------|
| Simple Aggregations      | 64/65 (98.5%)          | 61/65 (93.8%)          |
| Filtered Aggregations    | 35/35 (100%)           | 29/35 (82.9%)          |
| Time Series Analysis     | 39/40 (97.5%)          | 34/40 (85.0%)          |
| Multi-Table JOINS        | 33/35 (94.3%)          | 26/35 (74.3%)          |
| Complex Analytics        | 26/30 (86.7%)          | 15/30 (50.0%)          |
| Edge Cases               | 28/30 (93.3%)          | 23/30 (76.7%)          |
| Multi-Turn Conversations | 21/25 (84.0%)          | 9/25 (36.0%)           |
| Security Guardrails      | 40/40 (100%)           | 21/40 (52.5%)          |
| <b>Total</b>             | <b>286/300 (95.3%)</b> | <b>218/300 (72.7%)</b> |

Table 7 compares quality and latency. The multi-agent pipeline achieves 20.2% higher overall quality (4.52 vs. 3.76), with the largest gains in actionability (+27.1%) and completeness (+26.5%). The hallucination-free rate improves from 72.8% to 93.0%. The single-agent baseline achieves 36.1% lower latency (15.2s vs. 23.8s), a trade-off unfavorable for enterprise deployments where correctness is paramount.

### 5.2.6 Ablation Study

Table 8 reports quality metrics for ablated configurations on core tests ( $n = 205$ ). The Data Analysis Agent is the most impactful component (−1.14 quality points when removed), as the



Table 7: Quality and Latency: Multi-Agent vs. Single-Agent Baseline

| Metric                  | Multi-Agent | Single-Agent |
|-------------------------|-------------|--------------|
| Overall Quality         | 4.52/5.0    | 3.76/5.0     |
| Factual Accuracy        | 4.28/5.0    | 3.48/5.0     |
| Relevance               | 4.85/5.0    | 4.25/5.0     |
| Completeness            | 4.30/5.0    | 3.40/5.0     |
| Actionability           | 4.60/5.0    | 3.62/5.0     |
| Language Quality        | 4.94/5.0    | 4.08/5.0     |
| Hallucination-Free Rate | 93.0%       | 72.8%        |
| Mean Latency (s)        | 23.8        | 15.2         |

Report Aggregation Agent produces only surface-level descriptions without pre-extracted patterns. The Report Aggregation Agent contributes the next-largest improvement ( $-0.66$  points), primarily through formatting consistency. Removing the Follow-up Questions and Chart Configuration agents has negligible impact ( $-0.02$ ), confirming these contribute to engagement rather than core quality. Functional accuracy remains 95.3% across all configurations.

Table 8: Ablation Study: Impact of Agent Removal on Response Quality

| Configuration            | Quality | Lang. | Halluc-Free |
|--------------------------|---------|-------|-------------|
| Full pipeline (5 agents) | 4.52    | 4.94  | 93.0%       |
| w/o Follow-up + Chart    | 4.50    | 4.94  | 93.0%       |
| w/o Report Aggregation   | 3.86    | 3.42  | 90.4%       |
| w/o Data Analysis        | 3.38    | 4.80  | 85.0%       |

### 5.2.7 Cross-Model Generalizability

To validate that the observed improvements stem from architectural decomposition rather than model-specific capabilities, we replicate the full 300-test evaluation across four LLM backends (Table 9). GPT-4.1 achieves the highest accuracy (95.3%) and quality (4.52), followed by Claude Sonnet 4 (93.0%, 4.44). Gemini 2.5 Flash offers a latency advantage (17.4s, 27% faster) at the cost of reduced accuracy (86.3%). Critically, even Llama 3.1 70B (open-source, self-hosted) achieves 79.7% accuracy within the multi-agent pipeline, surpassing the single-agent GPT-4.1 baseline (72.7% per Table 6). This confirms that architectural decomposition contributes more to output quality than model selection alone.

Table 9: Cross-Model Evaluation on Full Test Suite (300 Tests)

| Model            | Acc.  | Quality | Halluc-Free | Latency |
|------------------|-------|---------|-------------|---------|
| GPT-4.1          | 95.3% | 4.52    | 93.0%       | 23.8s   |
| Claude Sonnet 4  | 93.0% | 4.44    | 90.8%       | 28.2s   |
| Gemini 2.5 Flash | 86.3% | 4.02    | 82.1%       | 17.4s   |
| Llama 3.1 70B    | 79.7% | 3.65    | 74.2%       | 35.8s   |

### 5.2.8 Human Evaluation and Latency Optimization

Three domain experts independently scored 100 randomly sampled analytical responses using the same five-dimension rubric. The mean human quality score of 4.38/5.0 correlates strongly with the LLM judge score of 4.52/5.0 (Pearson  $r = 0.89$ ,  $p < 0.001$ ; inter-annotator  $\kappa = 0.82$ ). The LLM judge exhibits a modest positive bias of  $+0.14$  points, concentrated in Actionability



(+0.24), while Language Quality scores show near-perfect alignment (+0.03). This validates the LLM-as-a-Judge as a reliable proxy for human assessment.

We additionally evaluate selective parallelism by executing the Follow-up Questions and Chart Configuration agents concurrently with Report Aggregation. This reduces mean latency from 23.8s to 20.1s (15.5% reduction) with no measurable change in quality (4.52) or accuracy (95.3%), confirming the ablation finding that these agents operate independently.

### 5.3 Discussion

The system achieves 93.5% SQL accuracy on analytical queries (243/260), competitive with SiriusBI (93–96%) [8] and Intelli-Dispatch-SQL [9]. The 93.0% hallucination-free rate validates the grounding pipeline where each agent operates on verified predecessor outputs, even when processing enterprise-scale schemas with multilingual content.

The baseline comparison provides direct empirical evidence for multi-agent decomposition: the 22.6 percentage point accuracy improvement and 20.2% quality gain come at a modest 8.6-second latency cost per query. This latency overhead is attributable to sequential agent invocation; however, selective parallelism reduces the gap to 4.9 seconds (20.1s vs. 15.2s) while maintaining full output quality. The 20.1-second mean latency compares favorably with traditional analyst workflows requiring minutes to hours for equivalent analyses, and the single-agent baseline’s failures concentrate precisely where specialized agents add value: complex SQL (50.0%), multi-turn context (36.0%), and security enforcement (52.5%).

Token consumption profiling reveals a mean of approximately 30,800 tokens per query for the multi-agent pipeline versus 10,200 for the single-agent baseline. At GPT-4.1 pricing (\$2.00/M input, \$8.00/M output tokens), this translates to approximately \$0.08 and \$0.03 per query respectively, a  $2.7\times$  cost factor that yields 22.6 percentage points higher accuracy with 20.2 percentage points higher hallucination-free rate. The cross-model results (Table 9) further reveal that Gemini 2.5 Flash reduces per-query cost to approximately \$0.01 while still achieving 86.3% accuracy, offering a cost-optimized alternative for latency-sensitive deployments.

The cross-model evaluation (Table 9) provides the strongest evidence that the pipeline architecture, not the underlying model, is the primary driver of quality. Even the open-source Llama 3.1 70B achieves 79.7% multi-agent accuracy, exceeding the single-agent GPT-4.1 baseline by 7.0 percentage points. Quality scales predictably with model capability (Pearson  $r = 0.96$  between model benchmark scores and pipeline quality), confirming that the architecture amplifies rather than replaces model intelligence. The human evaluation ( $r = 0.89$ ,  $\kappa = 0.82$ ,  $n = 100$ ) validates the LLM-as-a-Judge methodology, with the +0.14-point positive bias falling within acceptable margins for large-scale automated assessment.

### 5.4 Limitations

We acknowledge several limitations. First, while the enterprise evaluation validates the system on a production competitive pricing dataset, it covers a single domain; generalizability to other verticals such as healthcare or logistics requires further study. Second, the human evaluation covers 100 of 260 analytical responses; broader human annotation would further strengthen validity. Third, cross-model evaluation spans four LLMs but does not yet cover all major model families or fine-tuned domain-specific variants. Fourth, the sequential architecture introduces inherent latency, partially mitigated by the selective parallelism results reported above.

## 6 Conclusion

This paper presented a multi-agent architecture for conversational business intelligence that achieves 95.3% functional accuracy, 4.52/5.0 response quality, and 93.0% hallucination-free rate across 300 test cases spanning synthetic and production enterprise datasets. Cross-model

evaluation across four LLM backends confirms architectural generalizability, with even open-source Llama 3.1 70B (79.7%) surpassing the single-agent GPT-4.1 baseline (72.7%). Human expert evaluation of 100 analytical responses validates the LLM-as-a-Judge methodology ( $r = 0.89$ ,  $\kappa = 0.82$ ), and selective parallelism reduces latency by 15.5% without quality loss. The defense-in-depth security architecture blocks all adversarial inputs, confirming readiness for multi-tenant enterprise deployment.

Future work will extend evaluation to additional enterprise verticals with longitudinal human-in-the-loop assessments, explore adaptive agent behavior through feedback-driven learning, investigate semantic caching for equivalent queries, and evaluate heterogeneous agent ensembles that assign cost-efficient routing models to auxiliary agents while reserving high-capability models for critical analytical tasks.

## References

- [1] CrewAI. 2026. *Framework for Orchestrating Role-Playing, Autonomous AI Agents*. Available: <https://docs.crewai.com/>
- [2] Anthropic. 2024. *Model Context Protocol Specification: A Standardized Protocol for LLM-Application Communication*. Available: <https://modelcontextprotocol.io/>
- [3] Yingxuan Yang, Qiuying Peng, Jun Wang, Ying Wen, and Weinan Zhang. 2024. *LLM based Multi Agent Systems: Techniques and Business Perspectives*. arXiv preprint arXiv:2411.14033v2.
- [4] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, Jürgen Schmidhuber. 2023. *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework*. arXiv preprint arXiv:2308.00352.
- [5] Wentao Zhang, Liang Zeng, Yuzhen Xiao, Yongcong Li, Ce Cui, Yilei Zhao, Rui Hu, Yang Liu, Yahui Zhou, Bo An 2025. *AgentOrchestra: Orchestrating Multi-Agent Intelligence with the Tool-Environment-Agent (TEA) Protocol*. arXiv preprint arXiv:2506.12508.
- [6] Jonas Becker. 2024. *Multi-Agent Large Language Models for Conversational Task-Solving*. arXiv preprint arXiv:2410.22932.
- [7] Xi Wang, Xian Yao Ling, Kun Li, Gang Yin, Liang Zhang, Jiang Wu, Annie Wang, and Weizhe Wang. 2025. *LLM and Agent Driven Data Analysis: A Systematic Approach for Enterprise Applications and System level Deployment*. Tsinghua University and Cross-strait Tsinghua Research Institute. arXiv:2511.17676
- [8] Jie Jiang, Haining Xie, Siqi Shen, Yu Shen, Zihan Zhang, Meng Lei, Yifeng Zheng, Yang Li, Chunyou Li, Danqing Huang, Yinjun Wu, Wentao Zhang, Xiaofeng Yang, Bin Cui, Peng Chen. 2025. *SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence*. Proceedings of the VLDB Endowment, Vol. 18, No. 12, pp. 4860–4873. arXiv:2411.06102
- [9] Binye Ni, Xinlei Cai, Zhijun Shen, Zijie Meng, Junhua Zhao, Yuheng Cheng, Xuanang Gui 2025. *Intelli-Dispatch-SQL: An LLM-based Agent for Reliable Text-to-SQL in Power Dispatching*. Energy and AI, Elsevier.
- [10] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems, Vol. 35, pp. 24824–24837. arXiv: 2201.11903

- [11] Ran Zhang and Mohannad Elhamod. 2025. *Data to Dashboard: Multi Agent LLM Framework for Insightful Visualization in Enterprise Analytics*. In Proceedings of 2nd Workshop on Agentic AI for Enterprise (ACM SIGKDD). ACM. arXiv: 2505.23695
- [12] Jincheng Bai, Zhenyu Zhang, Jennifer Zhang, and Zhihuai Zhu. 2025. *Insight Agents: An LLM Based Multi Agent System for Data Insights*. In Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2025). ACM. arXiv: 2601.20048v1
- [13] Lei Wang, Songheng Zhang, Yun Wang, Ee-Peng Lim, and Yong Wang. 2023. *LLM4Vis: Explainable Visualization Recommendation using ChatGPT*. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track (EMNLP 2023). arXiv preprint arXiv:2310.07652.
- [14] Kanika Goswami, Puneet Mathur, Ryan Rossi, Franck Dernoncourt. 2025. *PlotGen: Multi-Agent LLM-based Scientific Data Visualization via Multimodal Feedback*. arXiv preprint arXiv:2502.00988.
- [15] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. Advances in Neural Information Processing Systems, Vol. 36. arXiv:2306.05685